

Linha de Pesquisa: Engenharia de Software

Tema Pretendido: Engenharia de Software apoiada por IA

Primeira Opção de Orientador: Prof. Jean Carlo Rossa Hauck

Segunda Opção de Orientador: Prof. Jônata Tyska Carvalho

Título: Detecção Automática de Feature Envy em Código-Fonte: Uma Abordagem Baseada em IA e Análise Semântica

Introdução: Os *code smells* são definidos como estruturas sintomáticas no código-fonte que, embora não sejam erros (bugs), indicam possíveis problemas de design mais profundos que podem levar a dificuldades de manutenção [1]. Dentre os diversos tipos, o Feature Envy é um *code smell* estrutural notório, que ocorre quando um método de uma classe utiliza mais dados ou funcionalidades de outra classe do que da sua própria [1]. Esta característica indica uma possível violação do princípio de encapsulamento e uma alocação inadequada de comportamento no sistema, levando a um alto acoplamento e baixa coesão. A presença deste *smell* está diretamente associada a uma degradação da qualidade do software, impactando negativamente a manutenibilidade e aumentando o esforço de manutenção [2].

A detecção deste *smell* é um desafio conhecido. As ferramentas de análise estática tradicionais (SonarQube, PMD, Checkstyle) representam a abordagem de status quo, baseando-se predominantemente em métricas quantitativas (ex: contagem de acessos) e na definição de limiares (*thresholds*) estáticos para identificar o *smell* [3].

Motivação: A principal limitação da abordagem puramente métrica, descrita acima, é a sua rigidez e a incapacidade de considerar o contexto de domínio, o que gera uma fonte significativa de imprecisão. Estudos demonstram que existe uma "lacuna de concordância" (*agreement gap*) substancial entre o que essas ferramentas reportam e a percepção de desenvolvedores experientes sobre o que de fato constitui um problema de design [8, 7]. Na prática, isso resulta em uma alta incidência de **falsos positivos** e **falsos negativos** [7].

Este cenário motiva a exploração de novas técnicas. A Inteligência Artificial, particularmente processamento de linguagem natural e aprendizado de máquina aplicados a código-fonte, oferece novas oportunidades para superar estas limitações através da análise semântica e contextual [5, 10]. Ao invés de depender apenas de contagens, modelos de IA podem aprender a "compreender" o contexto arquitetural, as relações semânticas entre entidades e as intenções de design subjacentes.

Problema de Pesquisa: Como desenvolver um algoritmo baseado em IA que detecte Feature Envy com maior precisão que as ferramentas existentes, reduzindo falsos positivos através da compreensão semântica do contexto de código?

Objetivos: O objetivo principal deste trabalho é desenvolver e validar um algoritmo baseado em IA e análise semântica que detecte Feature Envy com precisão superior às ferramentas atuais. A metodologia envolve, inicialmente, uma revisão sistemática do estado da arte e a medição do desempenho das ferramentas existentes para estabelecer uma baseline. Com base nessa análise, será consolidado um dataset focado nos pontos fracos dessas ferramentas (como falsos positivos), que servirá de base para propor e modelar uma nova abordagem híbrida (combinando métricas estáticas e semântica). Por fim, o modelo proposto será validado experimentalmente contra a baseline e os resultados serão publicados.

Discussão do Estado da Arte: As abordagens recentes para superar as limitações das ferramentas métricas [3, 7] têm se concentrado no uso de Inteligência Artificial [5]. O estado da arte nesta área evoluiu principalmente em duas frentes: modelos focados na estrutura do código, como Redes Neurais Gráficas (GNNs) [11], e modelos focados na semântica textual do código, como modelos de linguagem pré-treinados (PLM) [4, 6].

Contudo, o campo permanece polarizado e cada abordagem apresenta limitações. De um lado, as ferramentas puramente métricas continuam sofrendo com altas taxas de falsos positivos em padrões de design legítimos (como DTOs) por não compreenderem o contexto [7]. De outro, as abordagens puramente baseadas em IA também enfrentam desafios.

As abordagens focadas na estrutura (ex: GNNs), embora sejam eficientes em capturar relações complexas em grafos de código [11], ainda podem ter dificuldade em interpretar o contexto de domínio ou a intenção do desenvolvedor, levando-as a classificar padrões de design legítimos (como *Builders* ou *Factories*) como anomalias estruturais.

Similarmente, as abordagens focadas na semântica (ex: PLMs) também têm limitações. Embora demonstrem uma capacidade crescente de "entender" o código em nível textual [9], esta abordagem introduz um alto custo computacional. O *fine-tuning* completo destes modelos é tão custoso que pesquisas recentes focam em alternativas de "Parameter-Efficient Fine-Tuning" (PEFT) [10] para tornar a detecção viável. Além disso, mesmo com o *fine-tuning*, esses modelos podem ter dificuldade em capturar sinais quantitativos claros (ex: um método que acessa *dez* campos de outra classe), algo que as métricas tradicionais fazem trivialmente.

Portanto, a lacuna de pesquisa reside no desenvolvimento de uma abordagem híbrida, que combine a análise estrutural clássica (para identificar candidatos) com a compreensão semântica profunda (para validar o contexto), visando especificamente a redução de falsos positivos.

Possíveis Contribuições Científicas:

- Algoritmo inovador para detecção de Feature Envy com melhor precisão
- Dataset público anotado especializado em Feature Envy
- Análise sistemática das limitações das ferramentas atuais
- Metodologia híbrida (combinando análise estática e semântica) para a detecção de Feature Envy
- Artigo científico com resultados comparativos em conferência qualificada

Encaminhamento Metodológico: A metodologia da pesquisa está estruturada em cinco fases principais. A primeira fase (Meses 1-4) consiste na fundamentação teórica, através de uma revisão sistemática da literatura e de uma avaliação comparativa de ferramentas existentes, que servirá para estabelecer a baseline e identificar suas limitações. Na segunda fase (Meses 5-7), será construído o dataset, que envolve a seleção de projetos open-source, a anotação manual por múltiplos especialistas para garantir a confiabilidade e a categorização de casos limítrofes e falsos positivos. A terceira fase (Meses 8-12) foca no desenvolvimento do algoritmo, explorando uma abordagem híbrida que combina métricas tradicionais com features semânticas (como similaridade de embeddings), seguida da modelagem com técnicas de ML e validação cruzada. Na quarta fase (Meses 13-15), o algoritmo proposto será validado comparando-o com as ferramentas da baseline no dataset criado, utilizando métricas padrão (Precisão, Recall, F1-Score) e análise estatística para verificar a significância dos resultados. Finalmente, a quinta fase (Meses 16-18) será dedicada à disseminação, incluindo a finalização da documentação, a redação de artigos científicos e a submissão para conferências e periódicos qualificados.

Referências

- [1] Fowler, M. (2018). Refactoring: improving the design of existing code. Addison-Wesley Professional.
- [2] Sjøberg, D. I., Yamashita, A., Anda, B. C., Mockus, A., & Dybå, T. (2013). Quantifying the effect of code smells on maintenance effort. *IEEE Transactions on Software Engineering*, 39(8), 1144-1156.
- [3] Marinescu, R. (2004). Detection strategies: metrics-based rules for detecting design flaws. In *20th IEEE International Conference on Software Maintenance* (pp. 350-359).
- [4] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., ... & Zhou, M. (2020). Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*.
- [5] Allamanis, M., Barr, E. T., Devanbu, P., & Sutton, C. (2018). A survey of machine learning for big code and naturalness. *ACM Computing Surveys*, 51(4), 1-37.
- [6] Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., ... & Duan, N. (2022). Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366*.
- [7] Fontana, F. A., et al. (2015). An experimental evaluation of code smell detection tools. In *Proceedings of the 3rd Workshop on Software Visualization, Evolution, and Maintenance (VEM)*.
- [8] de Oliveira, D. B. M., et al. (2025). ImprovMLCQ: A Feature-Enriched Dataset for Advancing Code Smell Detection. In *Anais do XII Workshop Brasileiro de Software. Simpósio Brasileiro de Engenharia de Software*.
- [9] Hromadiuk, L. (2024). *Static Code Smell Analysis Using Large Language Models: An Empirical Study with Llama 3*. (Tese de Bacharelado, Universidade de Saarland).
- [10] Fernandes, E., et al. (2024). A Comprehensive Evaluation of Parameter-Efficient Fine-Tuning on Code Smell Detection. *arXiv preprint arXiv:2412.13801v2*.
- [11] Zhang, H., & Kishi, T. (2023). Long Method Detection Using Graph Convolutional Networks. *IPSJ Journal of Information Processing*, 31, 469-477.